

# Decision-based Sampling for Node Context Representation

Ikenna Oluigbo<sup>1</sup> and Hamida Seba<sup>1</sup> and Mohammed Haddad<sup>1</sup>

**Abstract**—Learning low dimensional representations requires an expressive technique capable of capturing the different features for nodes, the relationship between nodes in the network and thus their similarities. However, many existing embedding techniques focus only on capturing the structural patterns in the network by randomly sampling the nodes in the neighborhood of the target node. To deal with this issue, we propose DSNCR, a node representation framework which uses the non-linear node attributes as well as their neighbourhood structural information to capture nodes similarities. This approach computes a semi-supervised regression analysis on the node attributes to guide a flexible probability walk procedure, such that different neighbourhoods are explored to capture rich network attributes and structures in a learned embedding. We verify the effectiveness of our model on link prediction and node classification tasks using real-life benchmark datasets, for which our technique performs better than existing embedding methods.

## I. INTRODUCTION

Over the years, we have witnessed a massive explosion in the number of networks with various semantic and structural properties. Learning low dimensional representations for these networks requires an expressive technique capable of capturing the different features for nodes and the relationship between nodes in the network. We can typically classify embedding models into three categories :

- Structure preserving methods: These methods preserve the local neighborhood structure among directly or indirectly connected vertices of a network. Embedding methods such as LINE[1], GRAREP[7], WALKLETS[8], NODE2VEC[6], and DEEPWALK[2] belong in this category.
- Contextual attributes preserving methods: These methods leverages on attributes and contextual properties of a network to learn vector representations of nodes for specific learning tasks. These properties are preserved irrespective of the local or global position of vertices in the network. Such embedding methods includes ROLE2VEC[3], MUSAE[4] and ANRLBRW[5].
- Methods preserving both network structure and node attributes: These methods jointly learn embedding for both neighbourhood structure and node attributes. We can cite TriDNR [10] that considers network structures and node attributes to be strongly correlated.

A machine learning task requires algorithmic techniques capable of preserving in the learned embedding informative

sets of properties. For link prediction and structural visualization tasks, the embedding techniques should be capable of preserving inter-connectivity between nodes. Many techniques capture these features through hand-engineering or by solving some optimization problem (requiring a trade-off in balancing computational efficiency and predictive accuracy). A classification task requires efficiently capturing the contextual attributes about the graph granularity of interest.

In this paper, we present a learning framework capable of preserving the node's neighbourhood structure as well as the nodes features for attributed networks in low-dimensional vector embedding which can be visualized on embedding space. Our model takes an attributed network as input. First we aggregate the features of the nodes into a numerical value that will characterise each vertex using a node indexing technique used in graph databases [13]. The intuition is that each node, having aggregated the features of its surrounding neighbourhood, becomes a "database of several features" with unique roles. Then, our key contribution is to design a guided probability walk capable of capturing the neighbourhood cluster to which each node belongs as well as capturing nodes having similar features. To achieve this, we introduce a semi-supervised decision function into the probability walk which explores nodes' neighbourhood breadth-wise and intelligently aggregates nodes with the same unique attributes. The strength of such decision based walk is that there is control over the neighbourhood sample space such that the attribute of a source node is compared against those of every neighbourhood node to find definite similarities; which is more efficient in capturing node context compared to the random node selection method used in many existing embedding models [2], [8]. Figure 1 shows that our technique is able to model the neighbourhood topology from the input network, as well as aggregate nodes with similar attributes, irrespective of their location in the network. We evaluate the performance of our model with various real-life networks and experimented with link prediction and node classification tasks. Our contributions are as follows:

- 1) We propose DSNCR model, an approach which uses a semi-supervised decision function to learn low dimensional embedding for an attributed network, while preserving the neighbourhood structure and node's attributes properties.
- 2) We compare our model with existing state-of-the-art on node classification and link prediction tasks.

\*This work was supported by Petroleum Technology Development Fund, Nigeria with grant number PTDF/GFC/035, and Agence Nationale de la Recherche with grant number ANR-20-CE23-0002.

<sup>1</sup>Univ Lyon, UCBL, CNRS, INSA Lyon, LIRIS, UMR5205, F-69622 Villeurbanne, France. `firstname.lastname@univ-lyon1.fr`

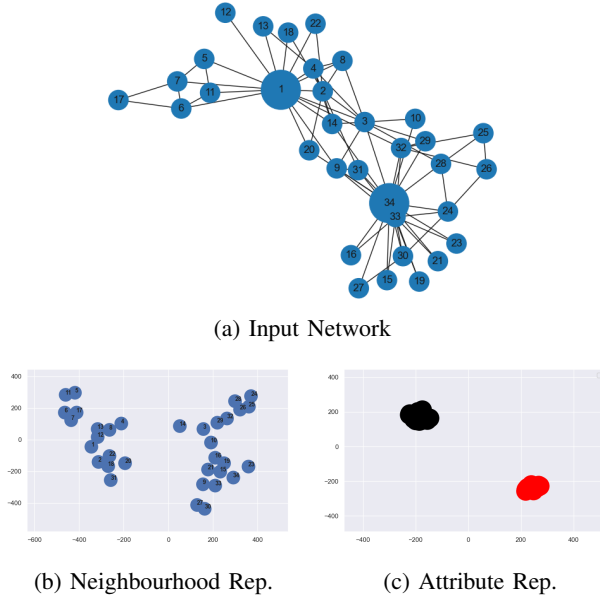


Fig. 1

The Input Network in figure 1a is the popular Karate dataset[11]. Our technique learn low-dimensional embedding of the network for both the neighbourhood structure (1b) and node attributes (1c). Note that the input network had members belonging in two categories of friendship.

## II. RELATED LITERATURE

In recent times, there's been a plethora of models for learning low latent representation of network data. These models aggregate network information in different ways. The popular means by which models extract network information is through the use of hand-engineered techniques such as extracting pair-wise adjacency matrix of a graph; this matrix is then decomposed into latent representations [8], [7]. The composition of this matrix does not take into account the rich attributes in the network. Recent studies as seen in [6], [2], [8], [10] used the random walk based approach with some defined tunable parameter, to extract patterns from input network. However, random walks are indeed randomized, missing out on certain information in the network. Another study, MUSAE[4], captures attribute-neighborhood relationships over multiple scales for latent feature identification across disconnected networks with similar features. We expand on the idea proposed in [13] and [15] that allows to compute an aggregated feature for a node using the attributes of its neighbors. This neighborhood aggregation was first studied in [13] where the authors proved how to encode all the information around a vertex in a single integer. In the study, the authors proposed an extremely efficient filtering technique for processing subgraph isomorphism search based on comparisons of the encoded integers. The study focuses mainly on querying a subgraph from a large graph. In [15], the authors adopted the concept of aggregated neighborhood to improve the node embedding derived from existing embedding models for downstream machine learning tasks. In our study, we took a different approach to using this aggregated feature. With the intuition that the aggregated

feature contains rich attributes from neighbourhood nodes, we applied a probability walk approach to learn embedding for nodes having similar features.

## III. DSNCR MODEL

### A. Problem Definition

We define the problem of embedding an attributed network in the context of node proximity in the network and their attributes.

*Definition 1:* An attributed Graph  $G$  is a 3-tuple  $G = (|V|, |E|, \ell)$ , where  $|V|$  consists of the set of vertices in the graph,  $|E|$  consists of the set of edges connecting the vertices in the graph, and  $\ell$  is a function associating attributes to vertices. The attributes describe the auxiliary information contained in nodes aside the structural relations of node pairs.  $\ell(u)$  represents the attribute of vertex  $u$  in a graph. Each edge in  $G$  is an unordered pair  $e = (u, v)$  with weight  $\omega_{uv} = 1$  demonstrating that the strength of relationships between all node pairs are equal.  $G$  is undirected such that  $(u, v) = (v, u)$  and  $\omega_{uv} = \omega_{vu}$ .

Given an attributed network  $G = (|V|, |E|, \ell)$ , for each node  $v \in |V|$ , our aim is to learn a low-dimensional vector representation in an embedding space  $Z_{(v_\ell)} \in \mathbb{R}^{V \times \delta}$ ; where  $\mathbb{R}^\delta$  is the embedding space with  $\delta$  dimensions.

To deal with this problem, our study is inspired by the skipgram model [12], which has found great success in the field of natural language processing. We model a probability walk from every node in the network to capture structural patterns and node contexts into a 'node corpus', for which we train using the skipgram model to maximize the probability of nodes in the same neighbourhood and nodes sharing similar attribute mapped closely in the embedding space. We start off by describing several components which builds up our model.

### B. Simplifying Node Properties Representation

To gather rich information about the diverse neighbourhoods in a network, we use a method borrowed from databases to index nodes according to their neighbourhood and introduced in [13]. This method aggregates, into a single feature, the neighborhood information surrounding a vertex using their attributes and also the surrounding structure of the node. To achieve this, it uses the following bijective function [14]:

$$g_k(x_1, x_2, x_3, \dots, x_k) = \sum_{j=1}^k h(j, x_1 + \dots + x_j) \quad (1)$$

where

$$h(p, s) = \binom{s+p-1}{p} = \frac{(s+p-1)!}{p!(s-1)!} \quad (2)$$

$\forall (x_1, x_2, x_3, \dots, x_k) \in \mathbb{N}^k$  and  $k > 0$ .

To use this bijective function to aggregate the neighborhood of nodes, the authors of [13] set the value of  $k$  to the number of distinct attributes in the network. To compute  $h(j, x_1 + \dots + x_j)$ ,  $j$  corresponds to a vertex attribute, and  $x_j$  is the number of apparition of attribute  $j$  in the direct neighborhood of the vertex.

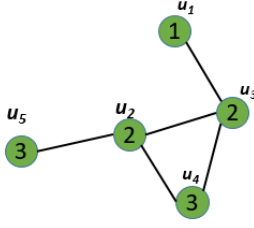


Fig. 2: Example for computing the aggregated features of nodes.

Hence, two vertices that have the same aggregated feature have similar neighborhoods. As an illustration for how to compute these aggregated values for each vertex using their attributes, let us consider the graph shown in Figure 2 with 5 vertices and 3 unique attributes (i.e.,  $k = 3$ ), represented by integers 1, 2, and 3. To compute the aggregated feature, for vertex  $u_1$ , we see that only attribute 2 is present in its neighborhood. Thus, the value of function  $g_k$  for vertex  $u_1$  is represented as  $g_3(0, 1, 0)$ . Therefore, the aggregated features (denoted as  $\Phi$ ) for vertices  $u_1$  to  $u_5$  from are computed as follows:

$$\begin{aligned}\Phi(u_1) &= g_3(0, 1, 0) = 2 \\ \Phi(u_2) &= g_3(0, 1, 2) = 11 \\ \Phi(u_3) &= g_3(1, 1, 1) = 14 \\ \Phi(u_4) &= g_3(0, 2, 0) = 7 \\ \Phi(u_5) &= g_3(0, 1, 0) = 2\end{aligned}$$

Clearly, vertex  $u_1$  and  $u_5$  have similar neighborhoods. It is therefore not a surprise they share the same value of  $\Phi$ .

### C. Sampling Similar Nodes

Random walks is predominantly designed to capture structures in a network. While this has proven to be successful in existing embedding models, such unguided random walks explore depths in the network far away from the source node (and its neighbourhood properties). As a result, the  $x_{n-1}th$  node in the walk  $x_0, x_1, x_2, \dots, x_{n-1}, x_n$  may not possess the same attributive properties as source node  $x_0$ . To deal with this issue, we propose a sampling technique with 2 steps; the first step captures the structural patterns of neighbourhood nodes, while the second step is aimed at capturing nodes sharing similar attributes. Sampling nodes in the network to capture these properties is achieved through a guided probability walk. Our model can then separately learn embedding for the neighbourhood structure of nodes, or for nodes sharing similar attributes as shown in Figure 1. In addition, we can learn representation which jointly preserves the neighbourhood structure and node attributes in the embedding space by concatenating the sampled node corpus into the skipgram model.

**a) Step 1: Preserving Node Attributes through Guided Probability Walk:** We propose a guided probability walk process for sampling sequence of nodes with similar attributes. To differ from conventional unguided random walks, we introduce a semi-supervised regression analysis decision function into the walk to maximize the likelihood for a

walk traversing from a node to a neighbour node with similar attributes. The regression analysis decision model measures the relationship between a node and its corresponding neighbours, and computes coefficient values for each neighbour node. Each coefficient value denotes to what extent the attribute of each neighbour node is strongly correlated to that of the source node. A neighbour node with similar attribute to a source node would show much higher positive correlative value. We use as attributes the aggregated features computed in the previous section. With these values, encoding neighborhood attributes and structure of nodes, we are able to explore these relationships at a higher scale. Assume a source node  $y$ , we apply the semi-supervised decision function to measure the relationship between node  $y$  and each node  $x \in N(y)$ . In this instance, since we are more concerned about correlations in the node attributes and not the neighbourhood topology, we found the regression analysis suitable as against other distance similarity metrics like Cosine distance, Minkowski distance, and Manhattan distance. The general expression of a multiple regression analysis is given as:

$$\hat{Y}_i = \hat{\beta}_0 + \sum_{i=1}^n \hat{\beta}_i \hat{X}_i \quad (3)$$

Starting from a source node  $y$ , and we wish to determine the transition probability of the walk traversing to another node  $x$  in the neighbourhood of  $y$ , we define the probability distribution:

$$Pr(\hat{x}|\hat{y}) = \underset{\hat{x}}{\operatorname{argmax}} - \hat{\beta}_0 + \sum_{i=1}^n \|\hat{x}_i \left( f_{\theta_i} \times \frac{\Phi_i}{n} \right)\| \quad (4)$$

where  $n$  is the number of nodes in the neighbourhood of  $y$ ,  $\Phi_i$  is the aggregated feature for node  $x_i$ . To represent the node attributes as a set of continuous quantities suitable for regression analysis, we multiply  $\Phi_i$  by a Gaussian distribution function represented as  $f_{\theta_i}$ ; with the length of the Gaussian distribution equal to the number of neighbors of each node instance  $x_i$ . In a bid to effectively capture the variations in the node neighbourhood attributes, the regression decision function first computes probability coefficients for each independent nodes  $x_i \in N(y)$  in relation to the source node  $y$  (dependent variable), each coefficient value describing the relationships to  $y$ , causing the walk to traverse to a neighbour node with the most relationship with  $y$  as determined by the highest coefficient value. This flexible decision walk process computationally determines a similar attributed node in neighbourhood of node  $y$ , which then assumes the next node in the probability walk. The relationship between the next node (which now bears similar attributes with the previous node) and its own neighbours are also computed, with the aim of identifying the next node with equally similar attribute. This chain of probability walk continues until each walk length is exhausted. Furthermore, the walk process ensures that every node in  $y$ 's neighbourhood is explored and compared with the attributes of  $y$ . In addition since every node in the network already possesses

information about its neighbour node with its aggregated feature, we are invariably ensuring that the attribute of the source node is compared with richer graph information as we explore farther depths in the network.

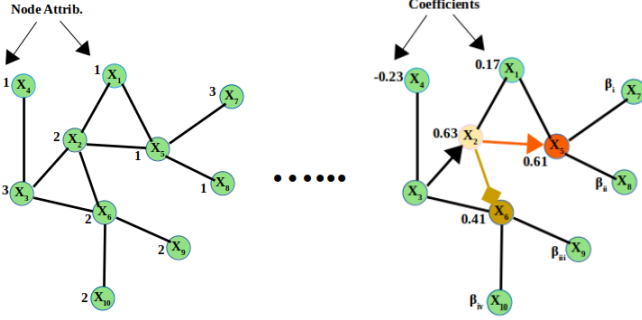


Fig. 3: Illustration of probability walk showing transition from node  $x_3$  through node  $x_2$  to node  $x_5$  (Attribute Preserving) and node  $x_6$  (Neighbourhood Structure preserving).

**b) Step 2: Preserving Neighbourhood Structural Patterns with Transition Bias.** Aggregating nodes with similar attributes across the network is made possible as a result of the semi-supervised decision function introduced into the probability walk. In preserving neighbourhood structure, how do we bias our walk transition to ensure that it is maintained within the neighbourhood structure of the source node? One simple way to achieve this would be to simply sample the neighbours of a source node and randomly select a node from the sampled neighbours. However this is not ideal because the walk can explore other unrelated neighbourhoods since it is random. Preserving the neighbourhood structure can be useful when reconstructing the original network.

Let us consider Figure 3 with source node  $x_3$  in a walk. In first step of the walk, the neighbours of node  $x_3$  are nodes  $x_4$ ,  $x_2$ , and  $x_6$ . We introduce the transition probability function of Equation (5) to traverse to the next node  $x_2$ . Now, the walk needs to take a decision amongst nodes  $x_1$ ,  $x_5$ , and  $x_6$ , such that the node in neighbourhood of  $x_3$  has the highest probability of getting sampled as part of the walk. We introduce two biases, the path length  $p$  and a penalizing constant  $c$ . We find the path length from source node  $x_3$  to all the neighbours of current node in the walk  $x_2$ , and then introduce the penalizing constant  $c$ . The goal of the penalizing constant is to create a large disparity in distance between  $x_3$  and neighbours of  $x_2$ , such that nodes with larger distance from  $x_3$  will be least considered as a potential next node in the walk. Therefore, the likelihood of the walk traversing to another node in close neighbourhood proximity of at most 2-hop distance to  $x_3$  becomes quite higher. Introducing this bias, the likelihood probability of sampling nodes in similar neighbourhood from Equation (4) becomes:

$$Pr(\hat{X}|\hat{y}) = \underset{\hat{X}}{\operatorname{argmin}} - \hat{\beta}_0 + \sum_{i=1}^n \left\| \hat{X}_i \left( f_{\theta_i} \times k \left( \frac{\Phi_i}{n} \right) \right) \right\| \quad (5)$$

Where  $k = p \cdot c$ . We aim to minimize the likelihood of visiting nodes which are more than 2-hops distance from source node ( node  $x_3$  in our example). From Figure 3, we see that the walk traversed to node  $x_6$  which is in the neighbourhood of  $x_3$ . However if we were interested in preserving nodes with similar attributes (and not neighbourhood topology) to  $x_3$ , the walk would traverse to node  $x_5$  instead with a higher coefficient value. By carefully simulating the probability walk, we can generate unique sets of walk samples with fixed walk length capturing the rich topology and context information in the network.

#### D. Learning Framework

Multiple representation learning options are possible in our proposed model. We can learn embedding to capture and preserve similar node attributes. Alternatively, we can choose to preserve a rich combination of attributes and neighbourhood topology in the learned embedding (Equation 5). Using Skipgram to train the node corpus sampled from the probability walk, our model is capable of jointly learning representations for neighbourhood structure and node attributes, which are preserved in the latent embedding space. The output from the skipgram model is a vector space consisting of defined dimensionality  $\delta$ , with each node in the corpus being assigned a corresponding vector in the space. Let  $N$  represents the node corpus derived from sampling node sequence through the probability walk, and  $Z$  is the entire feature matrix representations with size  $|V| \times \delta$  dimensions, the objective function which maximizes the log-probability of mapping source node  $u$  with its neighbours  $N(u)$  in the feature space through the set of similar vector representation is given as:

$$\max_Z - \sum_{n \in N} \left( |n| \cdot \sum_{i=1}^{|n|} \left( \sum_{x_i \in y=1}^{N(y)} \log Pr(x_i|Z(y)) \right) \right) \quad (6)$$

In Equation (6),  $|n|$  denotes the individual set of node sequence in the corpus, while  $Pr(x_i|Z(y))$  represents the feature relationship of source node  $y$  with a neighbourhood context node  $x_i$ . We optimize Equation (6) using stochastic gradient descent to learn vector embedding for each node in the network. The neighbourhood-source node pair probability relationship  $Pr(x_i \in N(y)|Z(y))$  of Equation (7) can be expressed as a softmax function of the product of their features:

$$Pr(x_i|Z(u)) = \frac{\exp(Z(x_i) \cdot Z(u))}{\sum_{u \in |V|} \exp(Z(v) \cdot Z(u))} \quad (7)$$

By optimizing Equation (7), the model maximizes the likelihood of predicting a neighbour node given a source node, thereby creating node embedding where nodes with similar properties have similar embedding. Equation 7 outputs a vector representation for each input node and the context nodes over a window size.

#### IV. EXPERIMENTAL EVALUATION AND RESULTS DISCUSSION

In this section, we evaluate the learned representation for DSNCR against LINE, DEEPWALK and NODE2VEC embedding methods, on two supervised learning task; Link Prediction and Node Classification.

We used the same set of parameters for the four models to sample nodes and train the node corpus. We adopt negative sampling for all models to approximate softmax probabilities. In addition, all models were optimized using Stochastic Gradient Descent (SGD). Other constant parameters include a dimension of 64, walk length of 80, number of walks 10, with a single epoch optimization.

TABLE I: Benchmark Datasets Used

| Dataset            | Nodes  | Edges  | Num. of Classes | Dataset Description     |
|--------------------|--------|--------|-----------------|-------------------------|
| Citeseer           | 3264   | 4536   | 6               | Scientific publications |
| Cora               | 2708   | 5429   | 7               | Scientific publications |
| FIRSTMM            | 56.6k  | 252k   | 5               | Biological and robotics |
| Proteins           | 43.5k  | 162.1k | 3               | Bioinformatics          |
| NCI-I              | 122.3k | 265.5k | 25              | Chemical Compound       |
| Enzymes            | 19.5k  | 75k    | 3               | Protein Structures      |
| DHFR               | 32k    | 67k    | 9               | Molecular Descriptors   |
| Political Retweets | 18.5k  | 61.2k  | 2               | Social Network          |

##### A. Link Prediction

Link prediction depicts how accurate missing or previously unseen links in a network are best predicted to be a part of the network, using information embedded in the learned representations. For our experiment, we considered all nodes and edges in the network, and randomly split our data in testing data (for evaluation) and training data (to train the link prediction model) in the ratio 30:70. We evaluated the results with some popular heuristic scores that have good performance in link prediction highlighted in Table II.

##### Result Discussion

We summarize the results for the link prediction. With the exception of LINE in Cora dataset, we observe that all models used in our experiment outperform the baseline heuristic scores in Table II. In seven (Cora, Proteins, Enzymes, DHFR, Politics, NCI, and Citeseer) out of the eight benchmark datasets, DSNCR outperforms NODE2VEC, DEEPWALK by up 4% in most cases, while our model outperforms LINE by about 32% in Cora network alone as seen in Table III. NODE2VEC proved quite competitive when predicting the edges in much denser FIRSTMM network, however our method outperformed other baseline methods across much sparsely distinct networks. Overall, the LINE model had the lowest prediction accuracy across all benchmark data set.

The higher prediction accuracy of our model can be attributed to the fact that our model took into account neighbourhood structure with a 2-hop distance of every node. The compact patterns preserved for each neighbourhood of nodes increases the likelihood of predicting missing links to a higher accuracy. Since the link prediction analysis involves predicting structural connection between nodes, it

makes sense that a technique capable of preserving the global structure of the graph by encoding successive neighbourhood information should perform well in such link predictive tasks.

TABLE III: AUC Evaluation Results for Link Prediction

| Datasets | NODE2VEC      | DEEPWALK | LINE   | DSNCR         |
|----------|---------------|----------|--------|---------------|
| Cora     | 0.7658        | 0.7529   | 0.5407 | <b>0.8027</b> |
| Proteins | 0.7478        | 0.7736   | 0.7103 | <b>0.7854</b> |
| FirstMM  | <b>0.8419</b> | 0.7464   | 0.7533 | 0.8417        |
| Enzymes  | 0.7419        | 0.7248   | 0.7403 | <b>0.7906</b> |
| DHFR     | 0.7730        | 0.7487   | 0.7124 | <b>0.8046</b> |
| Politics | 0.8365        | 0.8165   | 0.7538 | <b>0.8609</b> |
| NCI      | 0.8115        | 0.8278   | 0.7665 | <b>0.8305</b> |
| Citeseer | 0.7951        | 0.7301   | 0.7118 | <b>0.8020</b> |

##### B. Node Classification

For the node classification experiment, we used four benchmark datasets - Citeseer, Enzymes, FirstMM, Politics, with their description given in Table I. The labels in these datasets were assigned the features which we try to classify for each node. First, we learn node embedding for the attributed networks, and then we designed a classification model which takes the embedding as input. The model was trained with 200 epochs, early stopping on validation loss with 10 epochs of patience, and a categorical crossentropy loss function. We split the learned embedding into training and testing data; and for each dataset and embedding, we vary the training data from 20% to 60% and not beyond 60% to avoid overfitting the model. We evaluate the scalability of the results from our experiments using the F1-score metric citing how accurate the model classification is [9].

We see from the results in Figure 4 that as we progressively increased the training data, the results from Node2Vec and DSNCR were fairly similar. However, our model DSNCR outperformed Node2Vec in both politics and enzymes datasets. Of all four models, LINE performed poorly in terms of classification accuracy. This could be as a result of the unique way LINE samples vertices which does not allow nodes getting sampled in multiple neighbourhood they might belong in. DeepWalk on the other hand uses more randomness in sampling nodes during each walk, therefore there is a high chance of unwanted noise resulting from nodes with higher degrees. In addition, DeepWalk focuses majorly on learning structural patterns for a network with no attention paid to the rich node attributes, hence this explains its poor performance. From the weighted Macro F1 score, our model showed consistently a higher performance over other models. As the number of training data is increased, the rich node attributes learned and embedded in representations using DSNCR allows for a better classification result especially using 50% and 60% training data. Overall, we outperformed Deepwalk and LINE by almost 8% and 13% respectively.

#### V. CONCLUSION

This paper presents a novel approach for capturing the similarities between nodes in their low latent vector embeddings using rich neighbourhood structures and features in an attributed network. First, we aggregate the neighbourhood

TABLE II: Link Prediction Heuristic Scores

| Heuristic Method        | Mathematical Expression                               | Cora   | First MM | Protein | Enzymes | Political Retweet | DHFR   | NCI    | Citeseer |
|-------------------------|-------------------------------------------------------|--------|----------|---------|---------|-------------------|--------|--------|----------|
| Common Neighbour        | $ N(x) \cap N(y) $                                    | 0.6631 | 0.6843   | 0.7119  | 0.7481  | 0.7012            | 0.7567 | 0.7579 | 0.7602   |
| Jaccards Neighbour      | $\frac{ N(x) \cap N(y) }{ N(x) \cup N(y) }$           | 0.6580 | 0.6729   | 0.6932  | 0.6610  | 0.6521            | 0.6341 | 0.6737 | 0.7195   |
| Preferential Attachment | $ N(x)  *  N(y) $                                     | 0.6873 | 0.6891   | 0.7141  | 0.6783  | 0.7012            | 0.6862 | 0.7119 | 0.7028   |
| Adamic Index            | $\sum_{z \in N(x) \cap N(y)} \frac{1}{\log  N(z) ^2}$ | 0.7063 | 0.7168   | 0.7585  | 0.7705  | 0.7164            | 0.7521 | 0.7781 | 0.7478   |
| Resource Allocation     | $\sum_{z \in N(x) \cap N(y)} \frac{1}{ N(z) }$        | 0.6831 | 0.7045   | 0.6951  | 0.6859  | 0.7067            | 0.6872 | 0.7106 | 0.7119   |

x and y denotes nodes,  $N(x)$  and  $N(y)$  denotes the neighbour set of these nodes, while z is the common neighbour of node x and node y

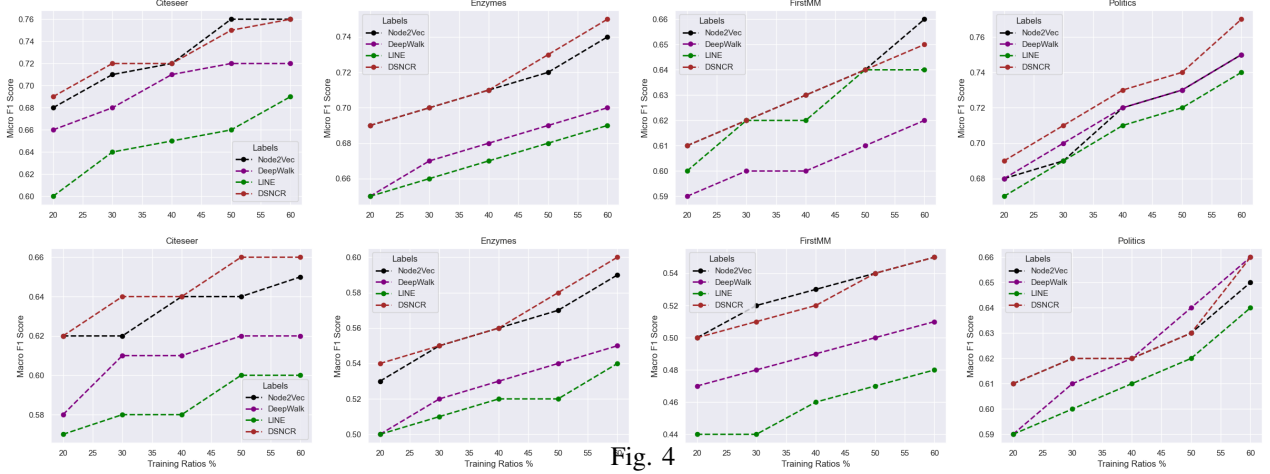


Fig. 4

Node Classification results at varied training sizes 20%,30%,40%,50%,60%. The x axis represents the Training ratio while y axis represents Micro F1 (top) and Macro F1 (bottom) score

information around a node into a single feature, and then we introduce a probability walk which follows a semi-supervised decision function, to prevent the "randomness" as seen in many other random walk sampling techniques. By so doing, our model can explore local node neighbourhood and global structures of a network without adding unwanted noise into the embedding. We showed the effectiveness of our model with real-life networks on link prediction and node classification tasks. In the future, we plan to investigate the application of our technique for embedding temporal sub-structures and detecting anomaly in a dynamic network.

## REFERENCES

- [1] Jian, T., Meng, Q., Mingzhe, W., Ming, Z., Jun, Y., Qiaozhu, M.: LINE: Large-scale Information Network Embedding. In: Proceedings of the 24th International Conference on World Wide Web, pp. 1067-1077. (2015).
- [2] Perozzi, B., Al-Rfou, R., and Skiena, S.: DeepWalk: Online Learning of Social Representations. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701-710. ACM (2014)
- [3] Ahmed, N. K., Rossi, R., Lee, J. B., Willke, T. L., Zhou, R., Kong, X., Eldardiry, H.: Learning Role-based Graph Embedding. J. ArXiv 1802.02896, pp. 1-8 (2018)
- [4] Rozemberczki, B., Allen, C., Sarkar, R.: Multi-scale Attributed Node Embedding. J. Complex Networks, vol. 9(2), pp. 1-8, (2021)
- [5] Dou, W., Zhang, W., Weng, Z.: An Attributed Network Representation Learning Method Based on Biased Random Walk. J. Procedis Comp Sc., vol. 174, pp. 291-298, (2020)
- [6] Grover, A., Leskovec, J.: node2vec: Scalable Feature Learning for Networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855-864. ACM (2016)
- [7] Cao, S., Lu, W., Xu, Q.: GraRep: Learning Graph Representations with Global Structural Information. In: Proceedings of the 24th ACM International on Conference on Information and Knowledge Management CIKM '15, pp. 891-900. ACM (2015). doi: 0.1145/2806416.2806512
- [8] Perozzi, B., Kulkarni, V., Chen, H., Skiena, S.: Don't Walk, Skip!: Online Learning of Multi-scale Network Embeddings. In: Proceedings of the 2017 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining ASONAM '17, pp. 258-265. ACM (2017). doi: 10.1145/3110025.3110086
- [9] Oluigbo, I., Haddad, M., Seba, H.: Evaluating Network Embedding Models for Machine Learning Tasks. In: Proceedings of International Conference on Complex Networks and Their Applications VIII, SCI 881, pp. 915-927. Springer (2019)
- [10] Pan, S., Wu, J., Zhu, X., Zhang, C., Wang, Y.: Tri-party deep network representation. J. Networks, vol. 11(9), pp. 12-18. (2016)
- [11] Ryan A.R., Nesreen K.A.: The Network Data Repository with Interactive Graph Analytics and Visualization. J. AAAI, (2015). networkrepository.com
- [12] Mikolov, T., Chen, K., Corrado, G.S., Dean, J.: Efficient Estimation of Word Representations in Vector Space. J. CoRR, vol. 1301.3781. (2013)
- [13] Nabti, C., Seba, H.: Compact Neighborhood Index for Subgraph Queries in Massive Graphs. <https://arxiv.org/abs/1703.05547> (2017)
- [14] Nathanson, M.: Cantor Polynomials and the Fueter-Pólya Theorem. The American Mathematical Monthly, vol. 123, pp. 1001 - 1012, (2015)
- [15] Oluigbo, I., Seba, H., Haddad, M.: Improving Node Embedding by a Compact Neighborhood Representation", Preprints university Lyon 1. <https://hal.archives-ouvertes.fr/hal-03638206>, (2022).